

Embeddings

How LLMs Understand Meaning Through Geometry

The Big Question: Yesterday tokens became integer IDs. Today: how does the model turn those IDs into **meaning**? The answer is embeddings — vectors that encode semantics as geometry. Similar meanings cluster close together in high-dimensional space.

Week 1 — Day Plan

Day	Date	Topic	Status
Day 1	Mon Jul 6	Tokenization	DONE
Day 2	Tue Jul 7	Embeddings	YOU ARE HERE
Day 3	Wed Jul 8	Transformer Architecture	Upcoming
Day 4	Thu Jul 9	Training Lifecycle	Upcoming
Day 5	Fri Jul 10	Scaling Laws + Prompt Engineering	Upcoming
Day 6	Sat Jul 11	Structured Output + Architect's Lens	Upcoming
Day 7	Sun Jul 12	Week 1 Project — Model Selection Dashboard	Upcoming

Topics Covered Today

#	Topic	Coverage
1	What is an Embedding?	Plain English — vector of floats encoding meaning
2	Token ID to Vector	The embedding layer: lookup table connecting Day 1 to Day 2
3	Geometry of Meaning	king – man + woman = queen; emergent arithmetic on vectors
4	Token vs Sentence Embeddings	What embedding APIs return; how RAG uses them
5	Similarity Metrics	Cosine similarity, dot product, L2 — when to use each

6	FAANG at Scale	Google Search, Netflix 260M users, Meta FAISS, Amazon product search
7	Embedding Model Comparison + MTEB	text-embedding-3-small, MiniLM, Cohere, BGE-M3 — cost, dims, MTEB scores, use case
8	Architect's Lens	3 decisions: no model mixing, token limits, cost at scale
9	Hands-On Exercise	sentence-transformers — cosine similarity matrix in Python

Section 1 — What is an Embedding? (Plain English)

Yesterday tokens became integers. Today those integers become **meaning**.

An **embedding** is a list of floating-point numbers — a vector — that represents a word, sentence, or document in a high-dimensional space. The model learns to place similar meanings **close together** in that space.

```
"dog" → [0.21, -0.84, 0.53, 0.11, ..., 0.67] (1,536 numbers)
"puppy" → [0.23, -0.81, 0.55, 0.09, ..., 0.65] (1,536 numbers)
"quantum" → [0.91, 0.33, -0.72, 0.88, ..., -0.21] (1,536 numbers)
```

"dog" and "puppy" are numerically close. "quantum" is far away. **Similarity in meaning = proximity in space.** This single idea powers Google Search, Netflix recommendations, Meta's content ranking, and every RAG system you will build in this course.

Section 2 — From Token IDs to Vectors: The Embedding Layer

Here is the exact path text takes inside a model — connecting Day 1 (Tokenization) to Day 2 (Embeddings):

```
Your text: "Netflix recommends movies"
↓ Tokenizer (Day 1)
Token IDs: [45231, 18, 7823]
↓ Embedding Layer ← TODAY
Vectors: [[0.21, -0.84, ...], [0.11, 0.73, ...], [0.53, -0.22, ...]]
↓ Transformer (Day 3)
Context-aware vectors (each word now understands its neighbours)
↓ Output head
Prediction / generation
```

The **embedding layer** is literally a lookup table: token ID → pre-learned vector. GPT-4 has a table of ~100,000 rows × 12,288 columns. That table has **1.2 billion numbers** in it — just for the first layer of the model.

Think of the embedding layer as a giant dictionary: you look up a word's ID and get back its vector. The difference from a regular dictionary is that these vectors encode relationship and meaning — not just definitions.

Section 3 — The Famous Equation: king – man + woman ≈ queen

This is not a metaphor. This is actual vector arithmetic that works on real embedding models:

```
vec("king") - vec("man") + vec("woman") ≈ vec("queen")
```

What this tells you: the model learned that the **difference** between "king" and "man" encodes the concept of *royalty without gender*. Adding the woman vector direction gives you "queen."

More real examples from production research:

```
Paris - France + Germany ≈ Berlin (capital city relationship)
Google - Search + Social ≈ Facebook (company + primary function)
Doctor - Hospital + Court ≈ Lawyer (professional + workplace)
walking - walk + swim ≈ swimming (verb tense transfer)
```

This emergent geometry is **not programmed** — it is *learned* from reading billions of documents. The model discovers that royalty, geography, profession, and grammar all have geometric structure. This is why embeddings are considered one of the most powerful ideas in modern AI.

Section 4 — Token Embeddings vs Sentence Embeddings

This distinction is critical for production system design:

Type	What it Embeds	Output	Used For
Token Embedding	Single token (word piece)	1 vector per token	Inside the model during generation
Sentence Embedding	Full sentence or paragraph	1 vector for the whole input	RAG, search, classification, clustering

When you call an embedding API, you are getting a **sentence embedding** — one vector summarising the entire input. This is what you will use in your RAG system in Weeks 3 and 4.

```
# Sentence embedding – 1 vector for the whole sentence
openai.embeddings.create(
  input='How does Netflix recommend movies?',
  model='text-embedding-3-small'
)
# Returns: 1 vector of 1,536 floats representing the full sentence meaning
```

Section 5 — Similarity Metrics: How You Measure 'Close'

Three metrics you will encounter in vector databases and search systems:

1. Cosine Similarity (most common)

Measures the angle between two vectors. Range: -1 to 1. Does not care about magnitude — only direction.

```
cosine_similarity(A, B) = (A · B) / (|A| × |B|)

"dog" vs "puppy" → 0.92 (nearly identical direction)
"dog" vs "cat" → 0.74 (related animals)
"dog" vs "finance" → 0.08 (unrelated)
```

Use when: embeddings may have different magnitudes (OpenAI, Cohere). This is the default for most RAG systems.

2. Dot Product

Raw multiplication — cares about both angle AND magnitude.

```
dot_product(A, B) = A · B = sum(Ai × Bi)
```

Use when: embeddings are normalised to unit length (magnitude = 1). Faster to compute than cosine. Used by OpenAI's internal systems and FAISS at Meta.

3. Euclidean Distance (L2)

Straight-line distance between two points in vector space. Lower = more similar.

$$L2(A, B) = \sqrt{\sum (A_i - B_i)^2}$$

Use when: magnitude differences carry meaning. Less common for language embeddings. More common for image embeddings.

Rule of thumb: Cosine for language embeddings. Dot product when embeddings are pre-normalised to unit length. Euclidean for image embeddings or when you need true geometric distance.

Section 6 — FAANG at Scale: How Embeddings Work in Production

Google Search — Semantic Search

Before embeddings, Google matched your query to exact keywords. With embeddings, searching "how to fix a slow database" finds results about "database performance optimization" even if those exact words aren't in your query. Google embeds both queries and documents; retrieval is nearest-neighbour search in vector space at **billions of documents per second**.

Netflix — Recommendation at 260M Users

Every movie and show gets embedded based on plot, cast, themes, pacing, genre. Every user's watch history creates a **user embedding** — a vector representing their taste. Recommendations = find movies whose embeddings are closest to the user's embedding. At 260M users x 15,000 titles, this runs as approximate nearest-neighbour search (ANN) using their internal vector search infrastructure.

Meta — FAISS (Facebook AI Similarity Search)

Meta open-sourced FAISS — the most widely used library for billion-scale vector search. It powers their content recommendation, ads targeting, and duplicate detection across billions of posts per day. You will use FAISS in Week 3 when you build your first Vector Search Lab.

Amazon — Product Search

Searching for "comfortable shoes for standing all day" returns orthopedic insoles, supportive sneakers, and gel inserts — products whose *descriptions* embed close to that *query*. No exact keyword match needed. Amazon's product catalogue has 350M+ items, all embedded and searchable semantically.

Section 7 — Embedding Models: What to Use When

Model	Dimensions	Cost	Best Use Case
text-embedding-3-small	1,536	\$0.02/1M tokens	Default for English RAG — best cost/quality balance
text-embedding-3-large	3,072	\$0.13/1M tokens	High-stakes retrieval where precision matters most
all-MiniLM-L6-v2	384	Free (self-hosted)	On-device, latency-sensitive, no API cost
all-mpnet-base-v2	768	Free (self-hosted)	Higher quality than MiniLM, still free, good default
Cohere embed-v3	1,024	\$0.10/1M tokens	Multilingual enterprise — best for 100+ languages
BGE-M3	1,024	Free (self-hosted)	Best open-source multilingual option

Dimension tradeoff: Higher dimensions = more expressiveness = better recall = more storage and slower search. A system with 10M documents at 1,536 dims uses ~59GB RAM. At 384 dims it uses ~15GB. Choose dimensions based on your recall requirements and infrastructure budget.

MTEB — How to Compare Embedding Models Objectively

The **Massive Text Embedding Benchmark (MTEB)** is the industry-standard leaderboard for comparing embedding models. It evaluates models across 56 datasets and 8 tasks: retrieval, clustering, classification, reranking, semantic textual similarity (STS), summarisation, bitext mining, and pair classification.

Model	MTEB Score	Dims	Verdict
text-embedding-3-large	64.6	3,072	Top API model — highest retrieval accuracy
text-embedding-3-small	62.3	1,536	Best cost-efficiency for most production RAG
Cohere embed-v3	64.5	1,024	Best multilingual; competitive with OpenAI large
BGE-M3 (BAAI)	63.6	1,024	Best open-source; free to self-host
all-MiniLM-L6-v2	56.3	384	Fast, lightweight — good for on-device/latency

Architect rule: Always check the MTEB leaderboard (huggingface.co/spaces/mteb/leaderboard) before choosing an embedding model for production. A model that looks good in a demo may rank poorly on retrieval tasks specifically. Filter by the task type that matches your use case.

Section 8 — Architect's Lens: 3 Embedding Decisions in Production

Decision 1 — Never Mix Embedding Models in the Same Index

If you embed your documents with text-embedding-3-small and later switch to text-embedding-3-large, the vectors are incompatible — they live in different geometric spaces. Comparing them gives meaningless similarity scores. Version your embedding model like a database schema. A migration requires re-embedding every document.

Rule: `embedding_model_version` is a schema property of your vector index.
Treat a model change as a breaking migration.
Always store which model generated each embedding alongside the vector.

Decision 2 — Input Length Limits Are a Hard Constraint

Embedding models have a maximum input token limit. Exceed it and the model silently truncates your text, losing information from the end. This is a silent failure — no error, just worse results.

text-embedding-3-small → max 8,191 tokens
all-MiniLM-L6-v2 → max 256 tokens (very short — fits ~2 paragraphs)
Cohere embed-v3 → max 512 tokens

If your document is 2,000 tokens and your model accepts 256,
you lose 87% of the content silently.
Chunking strategy (Week 3) is designed entirely around this constraint.

Decision 3 — Embedding Cost Compounds at Scale

Initial embedding is a one-time cost. But every time you add new documents, update content, or change your model, you pay again. At FAANG scale this dominates budgets.

Real example — Google-scale document store:
10 billion documents x avg 500 tokens x \$0.02/1M tokens
= \$100,000 just to embed the corpus once

With all-MiniLM-L6-v2 (free, self-hosted):
= GPU cost only (~\$2,000 one-time on 8xA100 cluster for 24 hours)

This is why Airbnb and Uber run their own embedding servers for high-volume use.

Section 9 — Hands-On Exercise: Cosine Similarity in Python

Run this exercise to see embeddings and semantic similarity in action. No API key needed — uses a free local model.

Setup:

```
cd /Users/thaya/AI-Engineering/week-01/notebooks
source /Users/thaya/AI-Engineering/.venv/bin/activate
pip install sentence-transformers # free, no API key needed
```

Create file: week-01/notebooks/day2_embeddings.py

```
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer('all-MiniLM-L6-v2')

sentences = [
    'Netflix recommends movies based on your watch history',
    'Amazon suggests products you might like',
    'Google finds relevant search results for your query',
    'The weather is sunny today',
    'Machine learning models learn from data',
    'Deep learning uses neural networks with many layers',
]

# Generate embeddings – each sentence → 1 vector of 384 floats
embeddings = model.encode(sentences)
print(f'Embedding shape: {embeddings.shape}') # (6, 384)

# Compute cosine similarity between all pairs
def cosine_sim(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print('Similarity matrix (pairs with similarity > 0.5):')
for i in range(len(sentences)):
    for j in range(i+1, len(sentences)):
        sim = cosine_sim(embeddings[i], embeddings[j])
        if sim > 0.5:
            print(f' {sim:.2f} | {sentences[i][:40]!r}')
            print(f' vs {sentences[j][:40]!r}')
```

What You Will Observe:

- Netflix/Amazon/Google sentences cluster together — all encode the concept of personalised retrieval
- ML and deep learning sentences score high similarity — semantically the same domain
- "The weather is sunny" is isolated — low similarity to all tech sentences
- Similarity is about meaning, not shared words — Netflix and Amazon share no common keywords but embed close
- Try adding your own sentences to build intuition for what 'close' means in vector space

Day 2 — Complete Summary

#	Topic	Key Takeaway	Status
1	What is an Embedding	Vector of floats; similar meanings cluster geometrically close in high-dim space	DONE
2	Token ID to Vector	Embedding layer is a lookup table; GPT-4's has 1.2 billion numbers	DONE
3	Geometry of Meaning	king – man + woman = queen; emergent arithmetic, not programmed	DONE
4	Token vs Sentence Embeddings	RAG uses sentence embeddings — 1 vector per chunk, not per token	DONE
5	Similarity Metrics	Cosine for language; dot product when normalised; L2 for images	DONE
6	FAANG at Scale	Google Search, Netflix 260M users, Meta FAISS, Amazon 350M products	DONE
7	Embedding Models + MTEB	text-embedding-3-small default; MiniLM free; Cohere multilingual; MTEB leaderboard for selection	DONE
8	Architect's Lens	No model mixing; respect token limits; cost compounds at scale	DONE
9	Hands-On Exercise	sentence-transformers cosine similarity matrix — week-01/notebooks/day2_embeddings.py	ACTION

Preview — Day 3: Transformer Architecture (Wednesday, Jul 8)

Now that you know tokens become vectors via the embedding layer, the next question is: **how does the model understand relationships between tokens?** How does it know that "bank" in "river bank" means something different from "bank" in "savings bank"?

The answer is the **Transformer architecture** — specifically the **self-attention mechanism**, the invention that changed everything in 2017 (the 'Attention Is All You Need' paper from Google).

Day 3 will cover:

- The Transformer architecture — encoder, decoder, encoder-decoder variants
- Self-attention: how every token looks at every other token to understand context
- Multi-head attention: why the model needs multiple 'perspectives' simultaneously
- Positional encoding: how the model knows word order without recurrence
- Feed-forward layers, residual connections, layer normalisation
- Architect's Lens: GPT vs BERT vs T5 — which architecture for which task

EAGLE EYE ADDENDUM — DAY 2 — EMBEDDINGS

Embedding History — How We Got Here

Modern embedding APIs did not appear overnight. Understanding the lineage from Word2Vec to transformer-based embeddings explains why current models work the way they do — and what problems each generation solved.

2013 — Word2Vec (Google, Mikolov et al.)

First practical dense word embeddings. Two architectures: CBOW (predict word from context) and Skip-gram (predict context from word). Trained on 100B Google News tokens. Produced 300-dimensional vectors. King - Man + Woman $\approx$ Queen. Fatal flaw: one vector per word — 'bank' (river) = 'bank' (finance).

CODE — Word2Vec (Google, Mikolov et al.)

```
from gensim.models import Word2Vec
model = Word2Vec(sentences, vector_size=300, window=5, min_count=1)
vector = model.wv["king"] # 300-dim float array
# Problem: model.wv["bank"] same vector for river bank AND financial bank
```

2014 — GloVe (Stanford, Pennington et al.)

Global Vectors for Word Representation. Word2Vec trains on local context windows; GloVe trains on global co-occurrence statistics across the entire corpus. Slightly better on analogy tasks. Same fatal flaw: static, one vector per word. Still used in NLP pipelines today for its simplicity and speed.

CODE — GloVe (Stanford, Pennington et al.)

```
import numpy as np
# Load pre-trained GloVe vectors (6B tokens, 400K vocab, 100/200/300d)
glove = {}
with open("glove.6B.300d.txt") as f:
    for line in f:
        parts = line.split()
        glove[parts[0]] = np.array(parts[1:], dtype=float)
vector = glove["netflix"] # 300-dim — same for all contexts
```

2016 — fastText (Meta, Bojanowski et al.)

Extends Word2Vec with subword information — each word represented as a bag of character n-grams. 'netflix' $\rightarrow$ ['net', 'etf', 'tfl', 'lix', 'netflix']. Key breakthrough: can generate vectors for out-of-vocabulary words. If 'netflixing' was never in training data, fastText uses its n-gram components. Still static (context-free), but handles morphology and new words.

CODE — fastText (Meta, Bojanowski et al.)

```
import fasttext
model = fasttext.load_model("cc.en.300.bin") # Common Crawl, 300d
vector = model.get_word_vector("netflixing") # Works even if OOV
```

```
# Uses n-grams: net + etf + tfl + flix + netfl + etfli + ...
```

The Static Embedding Problem

Word2Vec, GloVe, and fastText produce one fixed vector per word regardless of context. 'I went to the river bank' and 'I deposited money at the bank' — same vector for 'bank'. This is why transformer-based contextual embeddings (Day 2, Section 4) were revolutionary: every token gets a different vector based on its surrounding context.

Sentence-BERT (SBERT) — How Sentence Embeddings Work

BERT (2018) produces contextual token embeddings, but comparing two sentences required running them through BERT together — $O(n^2)$ complexity, unusable for large-scale search. Sentence-BERT (Reimers & Gurevych, 2019) solved this.

SBERT Architecture

SBERT fine-tunes BERT using Siamese networks — two identical BERT models with shared weights process two sentences in parallel. A pooling layer (mean of token embeddings) produces one fixed-size vector per sentence. Training uses Natural Language Inference (NLI) pairs to push similar sentence embeddings together and dissimilar ones apart.

```
SENTENCE-BERT — encode any sentence to fixed vector
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2') # 384-dim, fast
sentences = [
    'Netflix released a new sci-fi series',
    'A new science fiction show is available on Netflix',
    'Amazon stock price increased today',
]
embeddings = model.encode(sentences) # shape: (3, 384)
from sklearn.metrics.pairwise import cosine_similarity
sim = cosine_similarity(embeddings)
# sim[0][1] ≈ 0.91 (semantically identical)
# sim[0][2] ≈ 0.12 (unrelated)
```

Bi-Encoder vs Cross-Encoder — Critical Architecture Distinction

This is one of the most important practical distinctions in retrieval systems. Every RAG system (Week 3) uses both architectures in different stages.

	Bi-Encoder	Cross-Encoder
How it works	Encode query and each document SEPARATELY. Similarity = dot product of two independent vectors.	Concatenate query + document, run through model TOGETHER. Model attends across both texts.
Speed	Fast — precompute doc embeddings once. Query-time = 1 forward pass + ANN search. 10M docs in <100ms.	Slow — must run full model for every (query, doc) pair. 100 docs × 1 model pass = 100 passes.
Quality	Good — query/doc vectors computed independently, so cross-attention between query and document tokens is impossible.	Excellent — model sees full interaction between query and document tokens. Consistently outperforms bi-encoder by 5–15% on benchmarks.
Use case	Stage 1 recall: retrieve top-100 candidates from millions of documents.	Stage 2 re-ranking: rerank top-100 candidates to final top-10.

Examples	Sentence-BERT, text-embedding-3-large, BGE-M3, E5-large	ms-marco-MiniLM-L-12, Cohere Rerank, Jina Reranker v2
----------	---	---

Production Pattern — Bi + Cross Encoder Pipeline

Stage 1: Bi-encoder retrieves top-100 from 10M documents in <100ms. Stage 2: Cross-encoder re-ranks top-100 to top-10 in ~500ms. Final answer: top-10 with best quality. This two-stage architecture is used by Google, Amazon, and every serious RAG system.

Embedding Dimensions — Production Reference

Embedding dimension determines vector storage cost, retrieval speed, and quality. Every model makes a different trade-off. These are the numbers you need to know when designing a vector database or RAG system.

Model	Provider	Dimensions	Storage/1M docs	Best for
text-embedding-3-large	OpenAI	3072	12 GB	Highest quality English
text-embedding-3-small	OpenAI	1536	6 GB	Cost-balanced English
voyage-3-large	Anthropic/Voyage	1024	4 GB	Best code + multilingual
voyage-3	Anthropic/Voyage	1024	4 GB	Balanced production use
text-embedding-3-small (Matryoshka)	OpenAI	256–1536	1–6 GB	Dimension-flexible
BGE-M3	BAAI (Open)	1024	4 GB	Multilingual SOTA
all-MiniLM-L6-v2	SBERT (Open)	384	1.5 GB	Speed — lowest latency
all-mpnet-base-v2	SBERT (Open)	768	3 GB	Quality/speed balance

Anthropic's Voyage Embeddings

Anthropic offers embeddings through Voyage AI (acquired 2024). Voyage models are specifically tuned to work with Claude — they are trained on data that aligns with Claude's tokenization, making them the natural choice when building RAG systems that call Claude for generation.

```
VOYAGE EMBEDDINGS — Anthropic's embedding API
import anthropic

client = anthropic.Anthropic()

# Voyage embeddings via Anthropic SDK
# Note: requires ANTHROPIC_API_KEY with Voyage access
import voyageai
vo = voyageai.Client() # uses VOYAGE_API_KEY
result = vo.embed(
    ["Netflix quarterly earnings report Q4 2025",
     "Amazon Prime Video viewership data"],
    model="voyage-3-large",
    input_type="document" # vs "query" for retrieval queries
)

# result.embeddings[0] → list of 1024 floats
# input_type='document' vs 'query' optimises vectors for each role
```

Matryoshka Representation Learning (MRL)

OpenAI's text-embedding-3 models support MRL — you can truncate the 1536-dim vector to any smaller size (256, 512, 768...) and the truncated vector is still useful. This lets you store cheap 256-dim vectors for speed-critical paths and 3072-dim vectors for quality-critical paths — using the same model.

EAGLE EYE ADDENDUM — DAY 2: Sparse Embeddings & Hybrid Search

The Day 2 lesson covered dense embeddings (BERT, SBERT, bi-encoders, FAISS). One critical dimension was missing: sparse embeddings and hybrid search — the production standard for search at Google, Amazon, and Microsoft.

Sparse vs Dense Embeddings — The Full Picture

Dense Embeddings (Covered Day 2)

- Vector of 768 or 1536 floats — every dimension has a value
- Captures semantic meaning: 'car' and 'automobile' are close
- Produced by neural networks (BERT, SBERT, text-embedding-3)
- Weakness: poor at exact keyword matching and rare terms

Sparse Embeddings (New)

- Vector with 50,000+ dimensions (one per vocabulary token) — almost all zeros
- Only tokens that appear in the document get non-zero weights
- Exact keyword matching: 'PostgreSQL' and 'Postgres' are different terms
- Strength: rare technical terms, product codes, named entities, exact phrases
- Weakness: no semantic understanding — 'car' and 'automobile' are unrelated

BM25 — Classic Sparse (Term Frequency × Inverse Document Frequency)

BM25 is the standard sparse retrieval algorithm used by Elasticsearch, Lucene, and every classical search engine. It scores documents by how often query terms appear (TF) weighted by how rare the term is globally (IDF).

```
# BM25 score for a document d given query q:
# score(d, q) = sum over query terms t of:
#   IDF(t) * (TF(t,d) * (k1 + 1)) / (TF(t,d) + k1 * (1 - b + b * len(d)/avglen))
#
# IDF(t) = log((N - n(t) + 0.5) / (n(t) + 0.5))
# N = total documents, n(t) = documents containing t
# k1 = 1.5 (term saturation), b = 0.75 (length normalization)
#
# In Python (rank-bm25 library):
from rank_bm25 import BM25Okapi

corpus = [
    "SELECT user_id FROM orders WHERE amount > 100",
    "UPDATE users SET status = 'active' WHERE created_at > '2025-01-01'",
    "SELECT COUNT(*) FROM products GROUP BY category",
]
tokenized = [doc.lower().split() for doc in corpus]
```

```
bm25 = BM25Okapi(tokenized)

query = "select count group by"
scores = bm25.get_scores(query.lower().split())
# => array([0.0, 0.0, 2.3]) ← third doc matches best
```

SPLADE — Learned Sparse Embeddings

SPLADE (Sparse Lexical and Expansion model) uses a BERT encoder + MLM head to produce sparse vectors where weights are learned, not just term counts. It expands queries — given 'car repair', it also activates 'mechanic', 'vehicle', 'fix'. SPLADE matches BM25 on exact keywords while understanding synonyms.

```
# SPLADE produces a 30,522-dimensional vector (BERT vocab size)
# Most values = 0; activated tokens get learned importance weights
# Example for query "car repair":
{
  "car": 2.31, "vehicle": 1.87, "repair": 2.19,
  "mechanic": 1.42, "fix": 1.31, "automobile": 0.89,
  ... (30,500+ other tokens = 0.0)
}
# vs BM25 for same query: only "car" and "repair" activated
```

Hybrid Search — The Production Standard

Production search systems combine dense and sparse. Dense retrieval finds semantically similar content. Sparse retrieval finds exact keyword matches. Reciprocal Rank Fusion (RRF) merges the two result lists:

```
# Hybrid search with RRF (Reciprocal Rank Fusion):
def rrf_score(rank, k=60):
    return 1.0 / (k + rank)

def hybrid_search(query, dense_results, sparse_results, alpha=0.5):
    # dense_results: list of (doc_id, score) from FAISS
    # sparse_results: list of (doc_id, score) from BM25/Elasticsearch
    scores = {}
    for rank, (doc_id, _) in enumerate(dense_results):
        scores[doc_id] = scores.get(doc_id, 0) + alpha * rrf_score(rank)
    for rank, (doc_id, _) in enumerate(sparse_results):
        scores[doc_id] = scores.get(doc_id, 0) + (1-alpha) * rrf_score(rank)
    return sorted(scores.items(), key=lambda x: -x[1])

# alpha=0.5: equal weight (good default)
# alpha=0.7: trust dense more (semantic tasks)
# alpha=0.3: trust sparse more (keyword-heavy queries like SQL, code)
```

Method	Type	Exact Match	Semantic	Best For
TF-IDF	Sparse	Excellent	None	Legacy search, simple keyword tasks
BM25	Sparse	Excellent	None	Elasticsearch, Lucene, classical search
SPLADE	Sparse*	Excellent	Good	SQL/code search, technical terminology

SBERT	Dense	Poor	Excellent	Semantic similarity, RAG retrieval
Hybrid	Both	Excellent	Excellent	Production RAG, enterprise search

FAANG: Where Hybrid Search Runs

- Google Search: BERT dense + BM25 sparse + pagerank — all three combined
- Amazon: sparse (product codes, ASINs) + dense (semantic search) in product search
- Microsoft: Azure Cognitive Search uses BM25 + vector hybrid as default
- Meta: production RAG for LLaMA assistant uses hybrid FAISS + BM25

TEXT-TO-SQL CONNECTION: When retrieving schema information for SQL generation (Week 3 RAG), sparse retrieval is critical. Table names, column names, and SQL keywords (GROUP BY, HAVING, PARTITION BY) are exact terms — BM25 retrieves them reliably; dense-only retrieval misses them when the query uses different phrasing. Production schema RAG always uses hybrid.

EAGLE EYE ADDENDUM — DAY 2 | Vector Database Ecosystem

Day 2 covered dense embeddings, cosine similarity, BM25, SPLADE, and RRF hybrid search. One critical section was missing: the vector database ecosystem. Knowing HOW to create embeddings is useless without knowing WHERE to store and query them. This addendum covers the five main options every AI engineer must know.

The Vector Database Landscape

A vector database stores high-dimensional embeddings and enables approximate nearest neighbour (ANN) search at scale. Five categories cover the entire production landscape:

Name	Type	Best For	Scale
FAISS (Meta)	Library	Local ANN search; offline batch; research prototypes	Single machine
ChromaDB	OSS DB	Local dev, Python notebooks, RAG prototypes	< 1M vectors
Qdrant	OSS DB	Production OSS; filtering; Rust-based; fast	100M+ vectors
Weaviate	OSS DB	Hybrid search built-in; graph traversal; multi-modal	100M+ vectors
Pinecone	Managed	Zero-ops managed service; fastest time-to-prod	Billion-scale
pgvector	Postgres	Existing Postgres stack; low vector count; SQL + ANN	< 5M vectors
Milvus	OSS DB	High-throughput production; cloud-native K8s	Billion-scale

FAISS — The Research Baseline

```
import faiss, numpy as np

# Build a flat (exact) index for 1536-dim embeddings (text-embedding-3-small)
dim = 1536
index = faiss.IndexFlatL2(dim)

# Faster approximate search with IVF + quantization (production choice)
# IVF: Inverted File Index; 100 clusters; 8-byte product quantization
nlist = 100
quantizer = faiss.IndexFlatL2(dim)
index = faiss.IndexIVFPQ(quantizer, dim, nlist, 8, 8)
```

```
# Must train IVF index before adding vectors
sample = np.random.rand(50000, dim).astype("float32")
index.train(sample)
index.add(embeddings.astype("float32"))

# Search: k nearest neighbours
distances, indices = index.search(query_embedding, k=5)
# indices = row positions in your embedding matrix

# When to use FAISS:
# - Local research / offline batch indexing
# - Already using Python; no infra overhead needed
# - Not appropriate for multi-tenant or web-serving workloads
```

ChromaDB — The RAG Prototype DB

```
# pip install chromadb
import chromadb

client = chromadb.Client() # in-memory; use PersistentClient() for disk
collection = client.create_collection("documents")

# Add documents – Chroma embeds automatically with default model
collection.add(
    documents=["Machine learning is a subset of AI",
               "Neural networks learn from data"],
    ids=["doc1", "doc2"],
    metadatas=[{"source": "wiki"}, {"source": "textbook"}],
)

# Query by text (Chroma handles embedding)
results = collection.query(
    query_texts=["deep learning and neural nets"],
    n_results=2,
    where={"source": "wiki"}, # metadata filter
)
print(results["documents"])

# Use Chroma when: prototyping locally; experiments; < 100K docs
```

Qdrant — Production Open-Source

```
# pip install qdrant-client
from qdrant_client import QdrantClient
from qdrant_client.models import VectorParams, Distance, PointStruct

client = QdrantClient(":memory:") # or QdrantClient("localhost", port=6333)

client.create_collection(
    collection_name="articles",
    vectors_config=VectorParams(size=1536, distance=Distance.COSINE),
)

# Upsert with payload (structured metadata)
client.upsert(
    collection_name="articles",
    points=[
```

```

        PointStruct(id=1, vector=embedding_1.tolist(),
                    payload={"title": "Intro to LLMs", "date": "2024-01"}),
        PointStruct(id=2, vector=embedding_2.tolist(),
                    payload={"title": "RAG Architectures", "date": "2024-03"}),
    ]
)

# Filtered ANN search — Qdrant's key differentiator
from qdrant_client.models import Filter, FieldCondition, MatchValue
results = client.search(
    collection_name="articles",
    query_vector=query_embedding.tolist(),
    query_filter=Filter(must=[FieldCondition(
        key="date", match=MatchValue(value="2024-03")
    )]),
    limit=5,
)

# Use Qdrant when: self-hosted production; need metadata filtering

```

Pinecone — Managed Zero-Ops

```

# pip install pinecone-client
from pinecone import Pinecone, ServerlessSpec

pc = Pinecone(api_key="YOUR_API_KEY")
pc.create_index(
    name="articles",
    dimension=1536,
    metric="cosine",
    spec=ServerlessSpec(cloud="aws", region="us-east-1"),
)

index = pc.Index("articles")

# Upsert — batch for throughput
index.upsert(vectors=[
    {"id": "doc1", "values": embedding_1.tolist(),
     "metadata": {"title": "Intro to LLMs"}},
    {"id": "doc2", "values": embedding_2.tolist(),
     "metadata": {"title": "RAG Architectures"}},
])

# Query
results = index.query(
    vector=query_embedding.tolist(),
    top_k=5,
    include_metadata=True,
    filter={"title": {"$in": ["Intro to LLMs"]}},
)

# Use Pinecone when: managed infra; team has no Infra/K8s expertise;
# fastest time to production; cost-insensitive at small scale

```

pgvector — SQL + Vectors Together

```

-- pip install pgvector psycopg2-binary
-- In Postgres: CREATE EXTENSION vector;

```

```

CREATE TABLE documents (
  id      SERIAL PRIMARY KEY,
  title   TEXT,
  body    TEXT,
  embed   vector(1536) -- pgvector column
);

-- Insert embedding
INSERT INTO documents (title, body, embed)
VALUES ('RAG tutorial', 'Retrieval Augmented Generation...', '[0.12, -0.04, ...]');

-- Cosine similarity search (ANN via IVFFlat index)
CREATE INDEX ON documents USING ivfflat (embed vector_cosine_ops) WITH (lists = 100);

SELECT title, body, 1 - (embed <=> '[0.11, -0.03, ...]') AS similarity
FROM documents
ORDER BY embed <=> '[0.11, -0.03, ...]'
LIMIT 5;

-- <=> = cosine distance; <-> = L2 distance; <#> = inner product

-- Use pgvector when: already on Postgres; < 5M rows; SQL joins matter;
-- don't want a separate vector service

```

Decision	Choose
Building a local prototype in Python	ChromaDB (zero setup)
Production OSS, need metadata filtering	Qdrant
Production OSS, need hybrid + multi-modal search	Weaviate
Managed, no Infra team, fastest launch	Pinecone
Already on Postgres, < 5M vectors	pgvector
Offline batch, research, single-machine	FAISS
High throughput, Kubernetes, billion scale	Milvus

FAANG USAGE: Meta uses FAISS for offline embedding similarity (open-sourced 2017). Google uses ScaNN (Scalable Approximate Nearest Neighbor) — their internal FAISS equivalent. Amazon Bedrock Knowledge Bases uses OpenSearch with vector plugin (pgvector-style on OSS). Netflix uses Milvus for content recommendation feature stores. Microsoft Azure AI Search uses built-in vector search (HNSW index, Weaviate-style).